

Tecnológico de Monterrey

Instituto Tecnológico y de Estudios Superiores de Monterrey

Pablo Ramírez Zavala

Programación de Estructuras de Datos y Algoritmos Fundamentales

19/10/2026

Evidencia 2

Reporte Individual

In this report I explain the C++ program that my team made. The program reads a server log with 16,807 failed login attempts, saves them in a doubly linked list, sorts them by IP address and by time, and then shows every record inside an IP range that the user chooses. My part of the project was the range search and showing and saving the results. I also explain the data structures we used, when a linked list is a good choice for this problem and when it is not, how fast each step is in the best and worst case, and some ideas to make the program faster.

What Is the Problem, and Why Does It Matter?

A botnet is a group of computers that are infected and controlled by one attacker. Attackers often use botnets to guess passwords on servers: they try many passwords until one works [1], [2]. Every failed try is saved as one line in the server log, like this one:

```
Oct 9 10:32:24 423.2.230.77:6166 Failed password for illegal user guest
```

Our file, bitacora.txt, has 16,807 of these lines, and they are not in any order. This makes it hard to answer simple questions like “What did this IP do, and when?” or “Which attempts came from this group of addresses?”. Our task was to read the log, sort it by IP, ask the user for a start IP and an end IP, and then show and save all the records in that range. Records with the same IP had to be in time order [3].

This is useful because security teams usually study attacks by where they come from. They need to find a group of addresses quickly and see what happened, in order. Real logs can have millions of lines, so the data structure we choose changes how long this takes.

How Did We Solve the Problem?

The program follows a simple path: *read* the file, *split* each line into its parts, *save* it, *sort* it, *search* it and *write* the result [3]. We divided the work into four classes:

- **Entry**: stores one record. It also turns the month, day and time into one number (a timestamp), so comparing two dates takes only one step.
- **Keeper**: reads the file and saves each Entry in a doubly linked list.
- **Sorter**: sorts the records by IP. If two records have the same IP, it sorts them by time.
- **Searcher** (my part): asks for the IP range, finds the records inside it, shows them on screen and saves them in output.txt.

Figure 1 shows these steps. The main.cpp file calls them one after the other. For example, for the range 10.15.176.92 to 10.99.999.99 the program finds 14 records, and the whole program runs in less than 0.1 seconds.

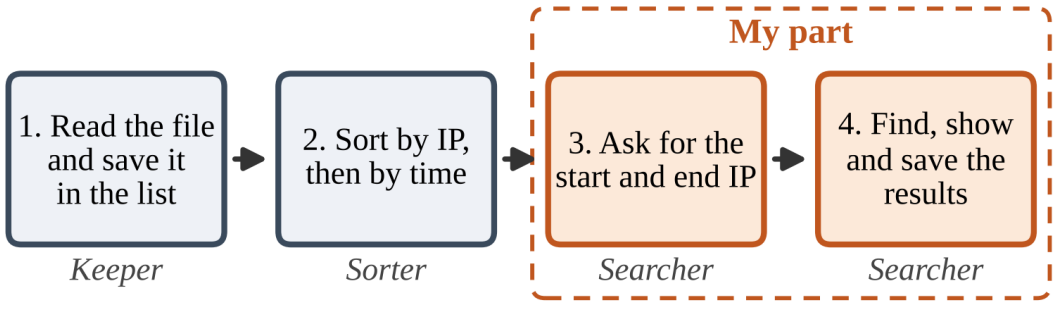


Figure 1. *Steps of the program. Steps 3 and 4 are my part.*

Data Structures and Algorithms

Doubly Linked List

A doubly linked list is a chain of nodes. Each node stores one Entry and two pointers: next points to the node after it, and prev points to the node before it. Keeper also remembers the first node (head), the last node (tail) and how many nodes there are. Because we always know where the tail is, adding a new record at the end (pushBack) takes the same short time no matter how long the list is. In Big-O notation this is O(1) [4]. We chose this structure because we do not know how many records the file has until we read it, and the list can grow one node at a time without copying anything.

Iterative Quicksort with a Stack

Quicksort picks one value, called the pivot, and moves the smaller values to its left and the bigger values to its right. Then it does the same thing again on each side. Usually this is written with recursion (a function that calls itself). We used a stack instead: the stack saves the start and the end of every part that still needs sorting. A loop takes one part from the stack, splits it around the pivot, and puts the two new parts back on the stack [3], [5]. This way the program does not need many recursive calls. Right now, our Sorter copies the records into a vector, sorts them using the middle element as the pivot and Hoare's method to split the data [6], and then copies them back into the list.

To compare two IPs, we turn each IP into one number. Each of the four parts of the IP is used as a "digit" in base 1000. We could not compare the IPs as text, because then "10.x" would come before "9.x". We also could not use base 256 (the normal rule for IPs), because this log has parts bigger than 255, such as 897.53.984.6.

Range Search (Searcher)

First, the program asks for the two IPs and checks that each one has four parts with one to three digits each. If the start IP is bigger than the end IP, the program swaps them instead of returning nothing. Both IPs are turned into numbers with the same base-1000 rule that the Sorter uses, so the search follows exactly the same order as the list.

Because the list is already sorted, all the records we want are next to each other. The search starts at head and does three things (Figure 2):

1) Skip every node whose IP is smaller than the start IP.
2) Mark the first node that is inside the range, and keep moving while the IP is not bigger than the end IP. Remember the last node inside the range.
3) Stop at the first IP that is bigger than the end IP. The rest of the list is never visited.

The program does not copy the results. It only keeps two pointers, first and last, and a counter. The functions display() and store() start at first, walk through that number of nodes, and print each record in the same format as the original file: Month Day Hour:Minute:Second IP:port message. The Sorter already put each IP in time order, so the output is also in time order without any extra work.

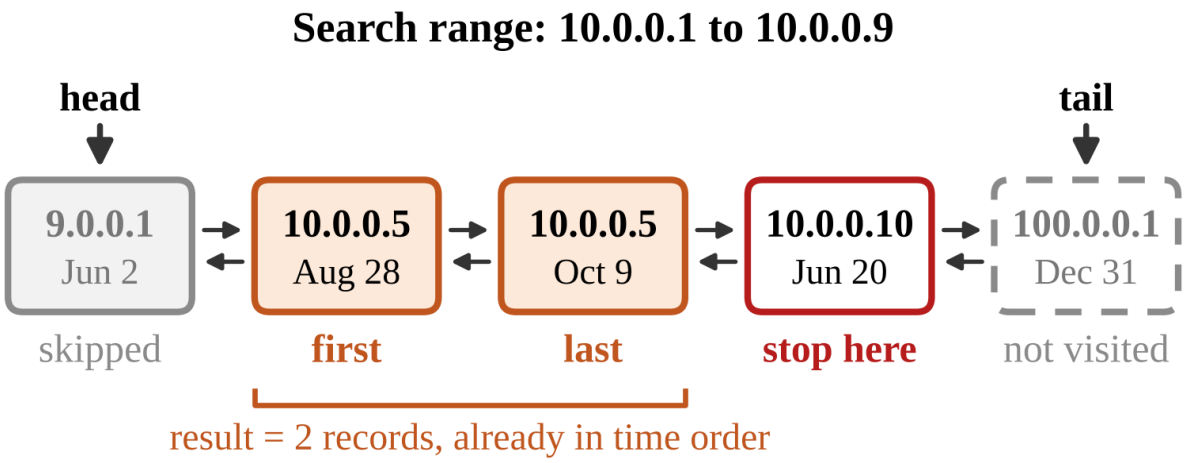


Figure 2. *How the search moves through the sorted list.*

Is a Linked List a Good Choice for This Problem?

Partly. It is a good choice for reading and saving the data and for returning the result, but not for sorting or searching.

Why it works well: we do not know the size of the file before reading it, adding a record at the end is O(1), and the search result can be a piece of the same list instead of a new copy. We can also remove or move records just by changing pointers.

Why it does not work well: we cannot jump directly to a position, like list[500]. To reach the middle, we have to walk there one node at a time. Because of this we cannot use binary search, so our search is O(n) in the worst case (it may visit every node). For the same reason, the Sorter had to copy the data into a vector. Also, every node uses extra memory for its two pointers, so a list uses more memory and is slower to walk through than an array.

With 16,807 records the difference is small. With millions of records, a sorted array or a search tree would be a better choice for searching.

Advantages and Disadvantages

Table 1. *Structures and algorithms we used*

Used for	Advantages	Disadvantages
Doubly linked list	Grows without copying data. Adding at the end is O(1). Can move forward and backward. The result can be part of the same list.	Cannot jump to a position. No binary search. Uses more memory per record.
Iterative Quicksort with a stack	Fast in most cases. No recursion. The middle pivot avoids the slow case when the data is already sorted.	Slow ($O(n^2)$) in the worst case. Needs to jump to positions. The stack can grow up to O(n).
Linear range search	Simple. Stops early. Needs almost no extra memory: O(1).	Slow, O(n), when the range is near the end of the list.

Best- and Worst-Case Complexity

Big-O notation tells us how the running time grows when the input gets bigger. O(1) means the time does not change with the size, O(n) means it grows at the same rate as n, and $O(n^2)$ means it grows much faster. In Table 2, n is the number of records in the log and k is the number of records found (k is never bigger than n).

Table 2. *Running time of each step*

Step	Best case	Worst case
Read and save the file (read, pushBack)	O(n)	O(n)
Turn the IP and the date into numbers	O(1)	O(1)
Sort (Sorter::sort)	O(n log n)	$O(n^2)$
Ask for and check the range	O(1)	O(1)
Search (Searcher::search)	O(k)	O(n)
Show and save the result	O(k)	O(k)
Whole program	O(n log n)	$O(n^2)$

Sort: the time depends on the pivot. If the pivot splits each part close to the middle, there are about log n levels of splitting, and each level does O(n) work, so the total is O(n log n). If the pivot is always the smallest or the biggest value, each step only removes one record, and the time grows to $O(n^2)$. Using the middle element as the pivot makes this very unlikely with our data.

Search: the time depends on where the start IP is in the sorted list. In the best case the range is at the beginning, so the search only visits the k records it finds, plus one more to stop. In the worst case the range is at the end of the list (or after the last IP), so it visits all n nodes.

Show and save: these always visit exactly k nodes. Overall, sorting is the slowest step, so in practice the whole program is O(n log n). For memory, the list and the copy used by the sort take O(n), and the search only needs O(1).

Possible Improvements

- Sort directly on the linked list, as we saw in class: swap the data inside the nodes and use a stack made from a singly linked list that saves node addresses [3]. This way we do not need to copy the data into a vector.
- After sorting, build an array of pointers to the nodes once. Then we can find the start of the range with binary search, and the search becomes O(log n + k).
- Use the prev pointer: if the start IP is closer to the end of the list, walk backward from tail.
- Save the IP as a number inside Entry, so the program does not have to calculate it again during the search.
- Let the user do several searches without reading and sorting the file again.

What I Learned

The most important thing I learned is that the order of the data matters as much as the data itself. Once the list was sorted, the search became a short walk that stops early, and the result was just two pointers. I also saw the main weakness of a linked list: because you cannot jump to a position, something that takes O(log n) in an array takes O(n) in a list. When choosing a data structure, you have to think about every operation the program needs, not only the first one.

References

[1] N. Hoque, D. K. Bhattacharyya, and J. K. Kalita, "Botnet in DDoS attacks: Trends and challenges," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 4, pp. 2242–2270, 2015.

[2] M. Javed and V. Paxson, "Detecting stealthy, distributed SSH brute-forcing," in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, Berlin, Germany, 2013, pp. 85–96.

[3] L. S. Flores Ugalde, "Problem situation: Bot War 2," class presentation, Programming of Data Structures and Fundamental Algorithms (TC1031), Tecnológico de Monterrey, 2026. Based on a presentation by V. Navárez Terán.

[4] M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed. Pearson, 2014.

[5] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.

[6] C. A. R. Hoare, "Quicksort," *The Computer Journal*, vol. 5, no. 1, pp. 10–16, 1962.